

AAKAAR

11-BROKER

Component: Rendezvous Broker

How agents and the platform pair when neither has a fixed address

Component

AUDIENCE	Engineers, Operations
VERSION	1.0
STATUS	Released
CLASSIFICATION	Confidential

Rendezvous Broker

In plain terms: imagine two people who both want to talk on the phone, but neither has a phone number you can dial — both are behind a switchboard that only lets them call *out*. The rendezvous broker is a meeting point with one fixed, dialable address that both sides call into. Once they have both called in, the broker simply patches their calls together and passes messages back and forth. It never reads the conversation, never decides who is allowed in (that is the API's job), and keeps no record once everyone hangs up.

In Aakaar, the two parties are a **remote agent** (running on a branch workstation) and the **Aakaar API** (running in your data centre). When neither has a network address the other can reach — both on DHCP, behind NAT, or on entirely separate networks — the broker is the one machine with a stable address that lets them find each other **by identity instead of by IP**.

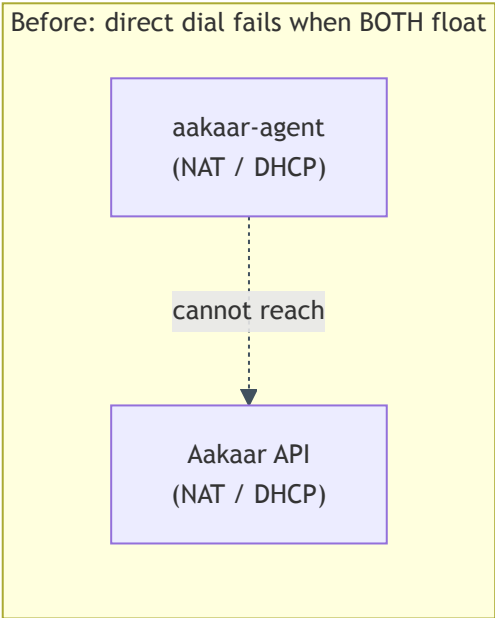
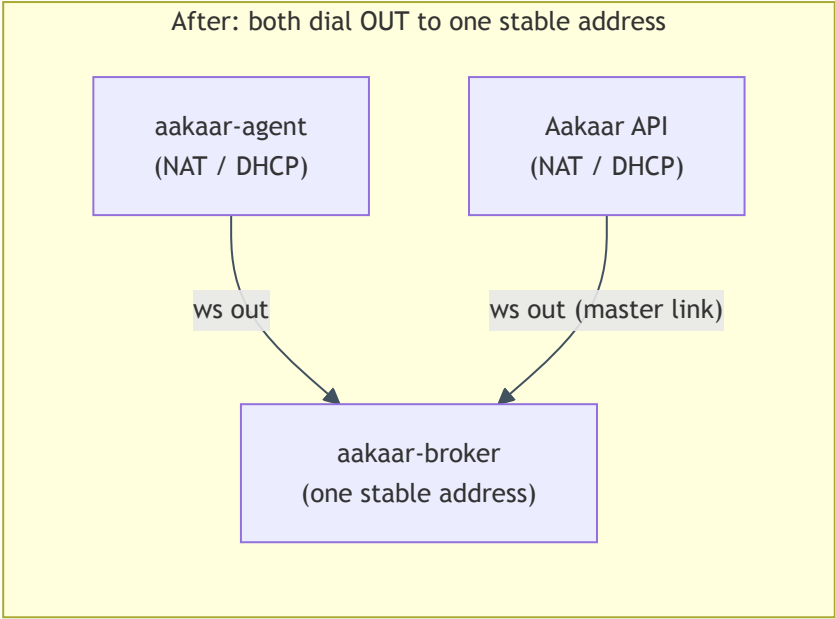
The broker is **optional and purely additive**. If your API already has an address the workstations can reach, you do not need a broker at all — agents dial the API directly. Direct connections keep working even when a broker is running.

The problem it solves

A remote agent never accepts inbound connections — it only dials *out* (so the workstation needs no open ports, no public IP). That is great for security, but it creates a chicken-and-egg problem: if the **API box** is *also* on a floating address (DHCP, NAT, a different network), then neither side has an address the other can dial. Direct connection is impossible.

The fix is a third party with one stable address that both sides dial out to.

Before vs after — where the dialable address lives:



Banking example: the API runs in a head-office VM that DHCP rennumbers on every reboot, and twelve branch reconciliation workstations sit behind branch-office NAT. No branch can dial the head office and the head office cannot dial any branch. A single small broker VM with a stable hostname (`broker.bank.internal`) lets all twelve agents and the API find each other.

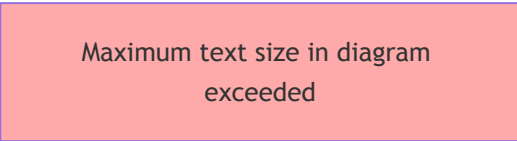
Topology — one master link, many multiplexed agents

The broker exposes exactly two WebSocket paths:

Path	Who dials it	Auth checked at the broker
<code>/ws/master</code>	the API (one connection)	<code>X-Broker-Token</code> header must equal <code>AAKAAR_BROKER_TOKEN</code>
<code>/ws/agents</code>	each agent (many connections)	none — the <code>X-Agent-Key</code> header is forwarded opaquely to the API

Every agent socket gets its own **session id** (`sid`). The broker announces each new session up the single master link and then relays text frames verbatim in both directions, multiplexed by `sid` . The API sees each relayed agent exactly as if it had connected directly — same `hello` /registration path, same end-to-end key check.

Topology of a broker-mediated fleet:



Maximum text size in diagram
exceeded